

CricMetrics Pro: Complete File-by-File Project Architecture Guide

Easy-to-Understand Blueprint for Every File in This Project

About this Document: This guide explains every single file and folder in the **CricMetrics Pro** project repository. For every file, it clearly explains in simple English:

- **WHAT** the file is.
- **HOW** it works inside.
- **WHEN** it is used in the project.
- **WHY** it is necessary.

1. Project Directory Structure

```
/Users/laxmanpatel/Desktop/ML
├─ app.py # Interactive Streamlit Web Application (6 Deliverable Tabs)
├─ requirements.txt # List of Python packages needed to run the project
├─ README.md # Project introduction and quickstart guide for GitHub
├─ FINAL_REPORT.md # Academic and viva voce project report (All 8 Deliverables)
├─ .gitignore # Tells Git which heavy files to ignore (like raw data)
├─ .streamlit/
│   └─ config.toml # Dark theme styling, unbuffered logging & performance config
├─ data/
│   └─ processed/
│       └─ cricket_players_clean.csv # Cleaned dataset of 619 cricketers with 32 calculated stats
│   └─ raw/
│       └─ matches_2008_2024.csv # 1,095 IPL match summaries (results, Player of the Match)
│       └─ deliveries_2008_2024.csv # 260,920 ball-by-ball records from 17 IPL seasons (69 MB)
├─ models/
│   └─ scaler.joblib # StandardScaler tool to normalize numbers (mean=0, std=1)
│   └─ linear_regression.joblib # Continuous baseline regressor ( $R^2 = 0.9490$ )
│   └─ polynomial_regression.joblib # Continuous non-linear regressor ( $R^2 = 0.9941$ , CV = 0.9956)
│   └─ random_forest_regressor.joblib # Production continuous rating engine ( $R^2 = 0.9780$ )
│   └─ knn_classifier.joblib # Best talent tier classifier (95.16% Test Accuracy)
│   └─ random_forest_classifier.joblib # Production tier classifier & Gini feature importance (94.35%)
│   └─ logistic_regression.joblib # Probabilistic tier classifier (87.90% Accuracy)
│   └─ decision_tree_classifier.joblib # Visual IF-THEN flowchart classifier (87.90% Accuracy)
│   └─ kmeans_model.joblib # K-Means model grouping players into 5 tactical styles
│   └─ pca_model.joblib # PCA tool compressing 29 stats into 2D coordinates (X, Y)
│   └─ feature_metadata.json # Column names, order, and tier labels
│   └─ metrics_summary.json # Test scores, cross-validation numbers, and confusion matrices
├─ notebooks/
│   └─ cricket_player_performance.ipynb # 10-cell executable Jupyter research notebook with full output
├─ src/
│   └─ cricket_data_pipeline.py # Script that cleans 260k balls into 619 player profiles
│   └─ train_cricket_models.py # Master script that trains, tests, and serializes all 10 models
│   └─ create_cricket_notebook.py # Automated generator for the reproducible Jupyter notebook
├─ docs/
│   └─ ML_MODELS_GUIDE.md & .pdf # Simple & easy guide to all ML models (What, When, Why, How)
│   └─ MACHINE_LEARNING_GUIDE.md & .pdf # In-depth guide to all ML topics and math formulas
│   └─ PROJECT_ARCHITECTURE_GUIDE.md & .pdf # File-by-file blueprint (This document)
```

2. File-by-File Explanations

2.1 Web Application Layer

`app.py` (Interactive Streamlit Dashboard)

- **WHAT:** The primary web application that sports executives, coaches, and scouts use to evaluate cricketers and simulate auction strategies (1,000+ lines of Python).
- **HOW:** Built with Streamlit, Plotly, and Pandas. It loads all trained models into memory once (`@st.cache_resource`) and displays 6 tabs mapped directly to project deliverables:
 1. 📖 *Problem Definition & Business Context*: Formulates the sports business problem, research questions, franchise constraints, and solution architecture.
 2. 📄 *Dataset & Preprocessing Pipeline*: Documents data sources (260,920 deliveries), data quality challenges, missing value handling, one-hot encoding, and standard scaling.
 3. 📊 *Exploratory Analysis (EDA) & Factor Impact*: Hexagonal skill radar charts (0–100), correlation heatmaps, role distributions, and Strike Rate vs. Average scatter plots.
 4. ⚡ *Live AI Valuation Engine*: Interactive sliders with 1-click presets (Virat Kohli, Jasprit Bumrah, Andre Russell, Sunil Narine, Hardik Pandya). It calculates continuous player rating (50–95), estimated auction purse (₹ Crores), talent tier probabilities, and tactical archetype.
 5. 🌿 *Tactical Archetypes & 2D PCA Latent Space*: 2D interactive PCA scatter map of all 619 players, colored by K-Means tactical style ($k=5$) or talent tier.
 6. 🏆 *Model Benchmarks, Evaluation & Viva Defense*: Full leaderboard comparing all regression and classification models, interactive Confusion Matrix heatmaps, Top 10 feature rankings, and oral viva Q&A defense.
- **WHEN:** Runs whenever you start the app locally (`streamlit run app.py`) or open the live deployment link in your browser.
- **WHY:** Bridges complex machine learning algorithms and real-world team decision-makers. Franchise directors need clear visual tools, not raw terminal commands.

`.streamlit/config.toml` (Theme & Server Settings)

- **WHAT:** Configuration file that styles and optimizes the Streamlit web application.
- **HOW:** Sets up a high-contrast dark theme with Slate Navy background (`#0F172A`) and Franchise Gold accents (`#F59E0B`), enables unbuffered info-level logging, and configures headless execution.
- **WHEN:** Loaded automatically by Streamlit before `app.py` starts.
- **WHY:** Ensures the app looks sleek, modern, and professional on all devices and browsers, avoiding generic browser defaults.

2.2 Data Pipeline Layer (`src/` and `data/`)

`src/cricket_data_pipeline.py` (Data Cleaning & Feature Engineering Script)

- **WHAT:** A Python script that reads 260,920 raw ball deliveries and aggregates them into clean player-level profiles.
- **HOW:**
 1. Reads `deliveries_2008_2024.csv` and `matches_2008_2024.csv` .
 2. Calculates batting stats: runs, balls faced, strike rate, boundaries, fifties.
 3. Filters balls in overs 16–20 to compute `death_overs_strike_rate` .
 4. Calculates bowling stats: overs bowled, wickets, economy rate, and `death_overs_economy` .

5. Counts Player of the Match awards for clutch resilience.
 6. Filters players with at least 3 matches or 20 balls faced/bowled, leaving **619 qualified cricketers**.
 7. Calculates 3 composite indices: Batting Impact, Bowling Impact, and Clutch Match-Winner Index.
 8. Saves the final clean table to `data/processed/cricket_players_clean.csv`.
- **WHEN:** Run whenever raw match records are updated (`python3 src/cricket_data_pipeline.py`).
 - **WHY:** Raw ball logs cannot be fed directly into machine learning models; they must first be converted into clean player summaries.
-

`data/processed/cricket_players_clean.csv` (Clean Analytical Dataset)

- **WHAT:** The final cleaned spreadsheet containing 619 qualified IPL cricketers and 32 columns (150 KB).
 - **HOW:** Contains 25 numerical metrics, player roles, continuous rating scores, discrete talent tiers, and estimated auction prices.
 - **WHEN:** Read by `app.py` and `train_cricket_models.py`.
 - **WHY:** Keeping this file small (150 KB) allows the cloud web app to start up in seconds, without needing to process the heavy 69 MB ball-by-ball file every time someone opens the website.
-

`data/raw/matches_2008_2024.csv` (IPL Match Records)

- **WHAT:** Summary of all 1,095 IPL matches played between 2008 and 2024 (218 KB).
 - **HOW:** Records match dates, teams, venues, match winners, and Player of the Match awards.
 - **WHEN:** Used by `cricket_data_pipeline.py` to extract clutch match-winner awards.
 - **WHY:** Player of the Match awards are official expert assessments of high-pressure impact.
-

`data/raw/deliveries_2008_2024.csv` (Raw Ball-by-Ball Data)

- **WHAT:** Every single delivery bowled in IPL history (260,920 rows, 69 MB).
 - **HOW:** Records batter, bowler, runs scored, extras, and wickets on every ball.
 - **WHEN:** Processed once by `src/cricket_data_pipeline.py`. Ignored by Git (`.gitignore`) to keep the repository lightweight.
 - **WHY:** Necessary to calculate situational metrics (like death overs strike rate in overs 16–20 and dot ball percentages) that standard scorecards do not show.
-

2.3 Machine Learning Layer (`models/` and `src/`)

`src/train_cricket_models.py` (Master Training & Evaluation Script)

- **WHAT:** Master script that trains, cross-validates, tests, and serializes all 10 Machine Learning models and transformers.
- **HOW:**
 1. Loads `cricket_players_clean.csv`.
 2. Applies One-Hot Encoding on `primary_role`.
 3. Splits data into 80% training (495 players) and 20% testing (124 players).
 4. Fits `StandardScaler` on training data and scales features.
 5. Trains and cross-validates 3 continuous regression models: Linear Regression (OLS), Polynomial Regression (Degree 2), Random Forest Regressor.

6. Trains and cross-validates 4 multi-class classification models: KNN ($k=7$), Random Forest Classifier, Logistic Regression, Decision Tree.
 7. Performs K-Means clustering ($k=5$) with Elbow and Silhouette scoring ($s=0.2676$).
 8. Fits PCA (2 components) for 2D tactical latent space.
 9. Saves all 10 trained joblib models into `models/` and exports scores to `metrics_summary.json` and `feature_metadata.json`.
- **WHEN:** Run during model development or retraining (`python3 src/train_cricket_models.py`).
 - **WHY:** Centralizes all model training and testing into one automated, reproducible script.

Serialized Model Files in `models/`

File Name	Model Type	Purpose in System	Benchmark Metric
<code>models/scaler.joblib</code>	StandardScaler	Rescales all 29 features to $\mu=0, \sigma=1$.	Eliminates scale dominance
<code>models/linear_regression.joblib</code>	Linear Regression (OLS)	Continuous rating baseline	$R^2 = 0.9490$, RMSE = 1.9534
<code>models/polynomial_regression.joblib</code>	Polynomial Regression (Deg 2)	Continuous curved fit	$R^2 = 0.9941$, CV = 0.9956
<code>models/random_forest_regressor.joblib</code>	Random Forest Regressor	Production rating & auction purse engine	$R^2 = 0.9780$, RMSE = 1.2837
<code>models/knn_classifier.joblib</code>	K-Nearest Neighbors ($k=7$)	Talent tier classification by historical peers	95.16% Test Accuracy, 0.9501 F1
<code>models/random_forest_classifier.joblib</code>	Random Forest Classifier	Production tier engine & Gini feature rankings	94.35% Test Acc, 94.75% 5-Fold CV
<code>models/logistic_regression.joblib</code>	Logistic Regression (Softmax)	Outputs tier risk probability distributions	87.90% Test Acc, 92.53% CV
<code>models/decision_tree_classifier.joblib</code>	Decision Tree (CART)	Visual IF-THEN flowchart for coaching audits	87.90% Test Acc, 89.90% CV
<code>models/kmeans_model.joblib</code>	K-Means Clustering ($k=5$)	Groups players into 5 tactical playing styles	$s = 0.2676$ Silhouette score
<code>models/pca_model.joblib</code>	Principal Component Analysis	Compresses 29 stats into 2D map (X, Y)	60.79% Total Variance Explained
<code>models/feature_metadata.json</code>	JSON Metadata	Stores column order, feature names, and tier labels	Ensures 100% pipeline consistency
<code>models/metrics_summary.json</code>	JSON Results	Stores test scores, CV scores, and confusion matrices	Feeds Tab 6 leaderboards in <code>app.py</code>

2.4 Research, Development & Documentation Layer

`notebooks/cricket_player_performance.ipynb` (Jupyter Research Notebook)

- **WHAT:** An interactive 10-cell data science notebook documenting the complete research study end-to-end.

- **HOW:** Contains executable code cells paired with Markdown explanations:
- *Cell 1:* Problem statement, student-formulated title, and business objectives.
- *Cell 2:* Dataset ingestion, inspection, and missing value checks.
- *Cell 3:* Exploratory Data Analysis, role distributions, and rating correlations.
- *Cell 4:* Data preprocessing, one-hot encoding, 80/20 train/test split, standard scaling.
- *Cell 5:* Continuous regression modeling (Linear, Polynomial, Random Forest) with 5-fold CV.
- *Cell 6:* Multi-class classification modeling (KNN, Random Forest, Logistic, Decision Tree).
- *Cell 7:* Detailed confusion matrix diagnostics.
- *Cell 8:* Feature importance analysis (regression vs classification factors).
- *Cell 9:* Unsupervised K-Means clustering ($k=5$) and 2D PCA latent space mapping.
- *Cell 10:* Academic conclusions, limitations, and viva voce defense.
- **WHEN:** Opened in Jupyter Notebook or VS Code for interactive experimentation and grading review.
- **WHY:** Gives professors, examiners, and data scientists a transparent, cell-by-cell walkthrough of our research.

`src/create_cricket_notebook.py` (Automated Notebook Generator)

- **WHAT:** Helper script that builds `notebooks/cricket_player_performance.ipynb` as pure JSON.
- **HOW:** Programmatically creates Markdown cells, executable Python code cells, and saves the notebook using `nbformat v4`.
- **WHEN:** Run whenever the research notebook needs to be refreshed (`python3 src/create_cricket_notebook.py`).
- **WHY:** Ensures the notebook can be regenerated reliably without manual copy-pasting.

`requirements.txt` (Python Package List)

- **WHAT:** Text file listing the exact Python libraries required to run the project.
- **WHEN:** Used when setting up the environment (`pip install -r requirements.txt`) and read automatically by Streamlit Cloud during online deployment.
- **WHY:** Guarantees that the code runs smoothly across Windows, Mac, Linux, and cloud servers.

`.gitignore` (Git Exclusion List)

- **WHAT:** Tells Git which files to ignore so they are not uploaded to GitHub.
- **HOW:** Ignores the `.venv/` virtual environment, Python cache files (`__pycache__/`), and the large 69 MB raw delivery log.
- **WHY:** Keeps the GitHub repository lightweight and prevents hitting GitHub's 100 MB file size limit, while keeping all clean datasets and trained models safely tracked.

`README.md` & `FINAL_REPORT.md` (Executive Project Reports)

- **WHAT:** The public GitHub homepage (`README.md`) and the comprehensive formal project defense report (`FINAL_REPORT.md`).
- **HOW:** Explains the project's background, quickstart commands, methodology proofs, and viva voce model answers aligned with all 8 syllabus deliverables.
- **WHEN:** Read by GitHub visitors, technical auditors, and academic evaluators.
- **WHY:** Clearly explains the business value of the project and satisfies the requirements of Case Study no. 102.